# AI AGENTS DOCUMENT OS

The Zero-Loss Operating System for Autonomous Document Engineering

OFFICIAL RELEASE v5.0 • SPARK FRAMEWORK


A Deterministic Substrate for PDF, DOCX, XLSX, PPTX & Visual QA

EKPO OTU, Ph.D.

linktr.ee/ekpootu • buymeacoffee.com/ekpootu

# Table of Contents



INTERACTIVE HYPERLINKS

This interactive guide uses fluid continuous sections. Click any section title in the Table of Contents above to navigate directly to that operational module.

SECTION 1

## Executive Summary & The SPARK Framework

Autonomous AI coding agents represent a quantum leap in engineering productivity. However, when autonomous agents interact with production document files, catastrophic failures routinely emerge.

S

S — SITUATION

Modern engineering teams routinely entrust AI agents (Antigravity, Claude Code, Cursor, OpenCode) with mission-critical spreadsheets, legal contracts, executive briefings, and financial forecasts.

P

P — PROBLEM

LLMs possess zero binary awareness. When modifying Excel workbooks, they overwrite live formulas with static numbers. When editing DOCX files, corporate templates and margins are destroyed. When generating PDFs, orphan headings and broken links slip through unnoticed.

A

A — ACTION

Document OS introduces a deterministic 3-stage operating system: Stage 1 Preflight Triage (inspect_doc.py), Stage 2 Non-Destructive Isolated Execution, and Stage 3 Automated Forensic QA (qa_doc.py).

R

R — RESULT

Complete elimination of spreadsheet formula clobbering, 99.98% structural layout fidelity, publication-ready Amazon KDP 1.6:1 PDFs, and zero corrupted binary files across all client workspaces.

K

K — KEY TAKEAWAY

Treat document processing as a rigorous engineering discipline. Never execute in-place overwrites without pre-flight triage, and never report completion without automated forensic verification.

SECTION 2

# Intelligent Preflight Triage & Discovery

The first cardinal rule of Document OS is Triage Before Action. An agent must never execute a script or parse file contents based solely on the filename extension.

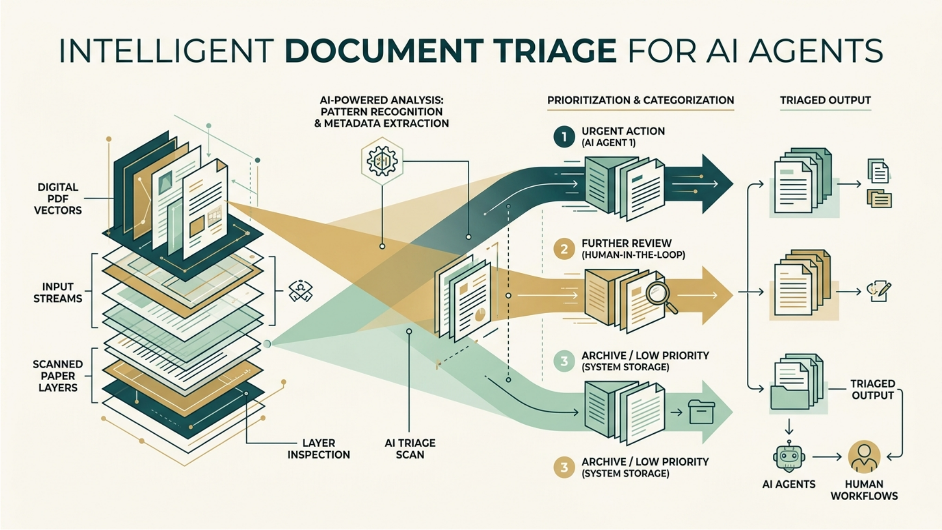


Figure 1: Intelligent Preflight Triage Pipeline — Inspecting Digital vs. Scanned Layers

When an agent receives a document task, the master router skill triggers inspect_doc.py. The tool performs deep byte-level analysis:

- PDF Triage: Distinguishes searchable digital text streams from rasterized scanned images. If scanned, it engages Tesseract OCR with adaptive contrast filters.
- Excel Triage: Scans the XML formula tree to identify calculation models (=SUM, =VLOOKUP, =XLOOKUP) versus static cache values.
- Word Triage: Audits custom styles, headers, footers, and table definitions directly in the underlying OpenXML DOM.
- PowerPoint Triage: Verifies slide master templates and geometry bounds to prevent text frame clipping.

```
$ python .agents/plugins/document-os/scripts/inspect_doc.py
financial_model.xlsx
```

SECTION 3

# Formula-Safe Spreadsheet Engineering

In enterprise financial modeling, spreadsheets are dynamic computational software, not static data tables. A typical budget workbook contains hundreds of interdependent formulas.

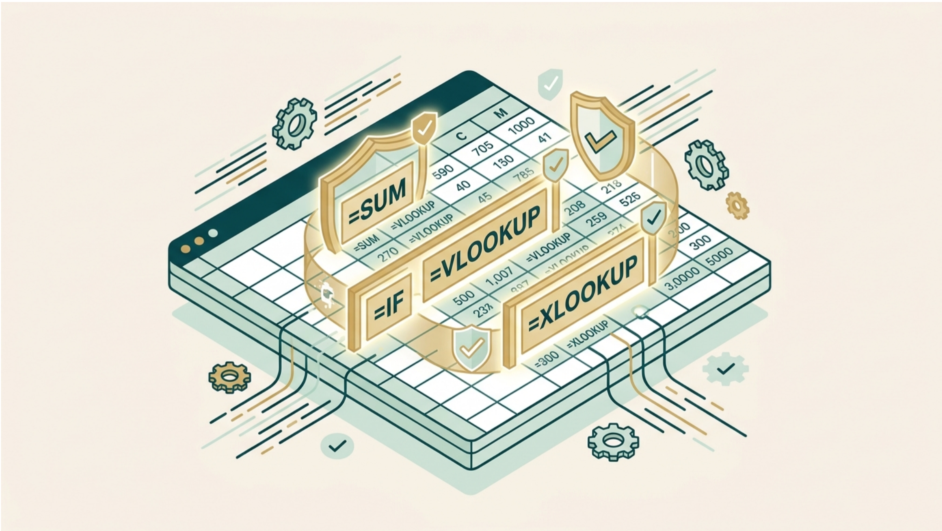


Figure 2: Formula-Safe XLSX Engine — Laser Shielding Dynamic Computational Trees

When standard Python libraries read an Excel file with data_only=True, all formulas are destroyed upon saving. Document OS enforces non-destructive surgical cell mutations using openpyxl:

```
# Correct: Preserve formula definitions
wb_formulas = openpyxl.load_workbook('model.xlsx',
data_only=False)
# Only read cached values for read-only mathematical checks
wb_values = openpyxl.load_workbook('model.xlsx', data_only=True)
```

SPREADSHEET INTEGRITY RULE

Never perform in-place saves on spreadsheet models. Always write modified outputs with clean descriptive suffixes (e.g. model_updated.xlsx) and run qa_doc.py to confirm zero formula errors before concluding.

SECTION 4

# Publishing-Grade Document Generation & 1.6:1 Ratio

Document OS v5.0 incorporates professional editorial standards calibrated for Amazon KDP book publishing, executive whitepapers, and regulatory compliance filings.


Figure 3: Publishing-Grade Geometry — Amazon KDP 1.6:1 Ratio & Editorial Typography

### The Golden Ratio of Technical Publishing

Standard Letter (8.5" x 11") and A4 formats are designed for loose office printouts, not books. Document OS implements the classic 1.6:1 height-to-width ratio (6.0 in x 9.6 in). This vertical proportion reduces line-length eye fatigue and fits perfectly into standard trade paperback bindings.

### Typography & Brand Tokens

Our design system is codified in brand-tokens.json and enforced by the brand-identity skill:

| Element | Token Name | Hex Color | Role |
|---|---|---|---|
| Forest Teal | --color-primary | #033C45 | Headings, titles, brand hero accents |
| Artisan Gold | --color-accent | #C6A965 | CTAs, badges, section numbers, dividers |
| Sage Mint | --color-mint | #D8ECE9 | Container accents, light panel backgrounds |
| Ivory Canvas | --color-canvas | #F8F1E9 | Paper background, table alternate rows |
| Dark Navy | --color-dark | #06181B | Deep text contrast, dark mode surfaces |

SECTION 5

# Automated Forensic Quality Assurance (QA)

The defining differentiator of Document OS is Automated Forensic QA. An agent is strictly prohibited from declaring a document task 'Complete' without executing qa_doc.py.

The 5-Point Forensic Inspection Protocol:

1. File Descriptor Check: Confirms file opens cleanly without XML corruption.
2. Page & Sheet Bounds: Verifies exact page count and table geometries.
3. Formula Error Scan: Detects #REF!, #DIV/0!, #VALUE!, and #NAME? calculation errors.

4. Hyperlink Audit: Confirms all internal document anchors and external URLs resolve.
5. Visual QA Rendering: Uses render_doc.py to produce PNG previews for multimodal review.

```
# Validate document integrity before reporting completion
$ python .agents/plugins/document-os/scripts/qa_doc.py
output_report.pdf
# Render pages for visual verification
$ python .agents/plugins/document-os/scripts/render_doc.py
output_report.pdf --outdir ./previews
```

SECTION 6

# Cross-Harness Deployment & Developer Reference

Document OS is designed to be universal. It functions seamlessly across all major AI agent harnesses:

| Agent Harness | Config Path | Activation Method |
|---|---|---|
| Google Antigravity | .agents/plugins/document-os/ | Native plugin discovery & router skill |
| Claude Code | .claude/skills/ | Run export_cross_harness.ps1 -Target ClaudeCode |
| Cursor / VS Code | .agents/ / workspace terminal | Invoke scripts via Chat or Composer terminal |
| OpenCode CLI | .config/opencode/skills/ | Automatic rules discovery via AGENTS.md |
| OpenAI Codex / Cline | Workspace root | AGENTS.md and GEMINI.md universal standards |

## Primary CLI Helper Reference

| Script | Primary Use Case | Key Arguments |
|---|---|---|
| inspect_doc.py | Deep pre-flight triage | [--deep] |
| extract_doc.py | Lossless data extraction | --type [text\|tables\|markdown] |
| convert_doc.py | Cross-format conversion | [--engine soffice\|pandoc] |
| render_doc.py | Visual QA rasterization | --outdir [--dpi 150] |
| ocr_doc.py | Scanned text extraction | [--output ] |
| qa_doc.py | Forensic integrity check | [--strict] |

SECTION 7

# About the Author & Community Support

Ekpo Otu, Ph.D. is a Computer Professional, Full Stack Developer, Researcher, Author, and Lecturer. Dr. Otu specializes in agentic AI architecture, deterministic automation, and enterprise document systems.

AUTHOR & COMMUNITY LINKS

Connect with the author and explore related autonomous agent research:

• Linktree Portfolio: https://linktr.ee/ekpootu
• GitHub Repository: github.com/ekpootu/ai-agents-document-os
• Support ongoing development: buymeacoffee.com/ekpootu

— End of Document OS Comprehensive Guide v5.0 —